UNIVERSIDAD CES · BASES DE DATOS · PROYECTO FINAL

# Guía de sustentación

Gestión de objetos perdidos: qué hace cada rol, lo que debes saber y cómo explicar dos reglas de negocio paso a paso


1. El sistema en un minuto
2. Los tres roles y qué puede hacer cada uno
3. Lo que debes saber (tecnologías, capas, base de datos, conceptos clave)
4. Regla de negocio 1: «Nada se borra»
5. Regla de negocio 2: «Recepción con fotos segura y completa»
6. Preguntas que te pueden hacer (con respuesta corta)
7. Chuleta de último minuto



Juan Alejandro Chaux Perdomo · Noviembre de 2026

Todas las credenciales que aparecen son ficticias, solo para desarrollo.

PARTE 1

# El sistema en un minuto

En las sedes de la Universidad CES (Poblado y Sabaneta) la gente encuentra objetos y los entrega en una oficina. El sistema registra **qué se recibió, quién lo entregó, dónde se guarda, quién lo reclamó y a quién se devolvió**, sin borrar nunca nada.

## La vida de un objeto

**1. Recibir** Alguien lo entrega; se registra con fotos → **2. Custodiar** Queda guardado y DISPONIBLE → **3. Reclamar** El dueño dice «es mío» → **4. Verificar** Se revisa la identidad y se acepta o rechaza → **5. Devolver** Queda DEVUELTO y sale una constancia PDF

**Dos «interruptores» distintos en cada objeto**

**status** dice dónde está el objeto: `DISPONIBLE` (lo tenemos) o `DEVUELTO` (ya lo entregamos).
**is_active** dice si el registro es válido: `true` (bien registrado) o `false` (se inactivó porque era un error, con motivo).
Ejemplo: un objeto devuelto sigue *activo*, porque es historia real. Uno registrado dos veces por error se *inactiva*, pero no se borra.

## Cómo lo construimos

| Parte | Tecnología | Para qué |
|---|---|---|
| Base de datos | PostgreSQL (15 tablas) | Guardar todo con reglas de integridad |
| Backend (API) | Python + Flask, SQLAlchemy, Flask-Migrate, JWT | La lógica: quién puede hacer qué y cómo |
| Frontend | Angular | Las pantallas que usa la gente |
| Pruebas | Postman (253 solicitudes) y pytest (55 pruebas) | Demostrar que todo funciona, incluso los errores |

**Resultados que puedes mostrar (probados en tu computador)**

Postman: **781 de 781** pruebas aprobadas, 0 fallidas · pytest: **55 passed** · Concurrencia: 10 devoluciones simultáneas → **una 201 y nueve 409** · Login con los tres roles en el frontend.

PARTE 2

# Los tres roles

Piensa en una oficina de objetos perdidos: el **jefe** (ADMIN), el **empleado de ventanilla** (OPERADOR) y el **visitante que pregunta** (CONSULTA).

ADMIN

### El jefe

- Hace todo lo del operador.
- Maneja usuarios y les cambia el rol.
- Crea y edita sedes, bloques, dependencias, categorías.
- Reactiva lo inactivado.
- Corrección administrativa de objetos devueltos.
- Ve la auditoría y el dashboard.

OPERADOR

### El de ventanilla

- Recibe objetos con fotos.
- Edita objetos disponibles (con motivo).
- Registra personas, reclamaciones y devoluciones.
- Inactiva errores (con motivo), pero no reactiva.
- Crea tipos de objeto nuevos.
- Ve inventario y lo exporta en PDF.

CONSULTA

### El visitante

- Solo mira, no modifica nada.
- Ve objetos **activos y DISPONIBLES**.
- Ve solo: código, tipo, categoría, sede, fecha y estado.
- **No** ve fotos, descripción, ubicaciones ni datos de personas.
- Puede cambiar su propia contraseña.

**¿Cómo se obtiene cada rol?**

Quien se registra solo en la página **siempre** queda como CONSULTA (aunque envíe «role: ADMIN», el servidor lo ignora). Un ADMIN lo puede promover a OPERADOR. El primer ADMIN lo crea el seed ( `admin@ces.test` ).

## Tabla de permisos (lo que dice el código del backend)

| Acción | ADMIN | OPERADOR | CONSULTA |
|---|---|---|---|
| Registrarse, iniciar sesión, recuperar y cambiar contraseña | Sí | Sí | Sí |
| Ver catálogos (sedes, bloques, dependencias, categorías, tipos) | Sí | Sí | Sí |
| Crear / editar / inactivar / reactivar sedes, bloques, dependencias y categorías | Sí | No | No |
| Crear un tipo de objeto nuevo | Sí | Sí | No |
| Editar, inactivar o reactivar un tipo de objeto | Sí | No | No |
| Ver objetos | Todo | Todo | Reducido |
| Recibir un objeto (con fotos) | Sí | Sí | No |
| Editar un objeto disponible / inactivarlo con motivo | Sí | Sí | No |
| Reactivar un objeto / corrección administrativa | Sí | No | No |
| Ver fotos, agregar fotos, inactivar una foto | Sí | Sí | No |
| Reactivar una foto | Sí | No | No |
| Ver, crear y editar personas | Sí | Sí | No |
| Inactivar o reactivar personas | Sí | No | No |
| Registrar, editar y decidir reclamaciones (aceptar / rechazar) | Sí | Sí | No |
| Inactivar o reactivar reclamaciones | Sí | No | No |

| | | | |
|---|---|---|---|
| Registrar devoluciones y descargar la constancia PDF | **Sí** | **Sí** | **No** |
| Inventario y exportación PDF | **Sí** | **Sí** | **No** |
| Historial de cambios de un objeto | **Sí** | **Sí** | **No** |
| Auditoría general, usuarios, roles y dashboard | **Sí** | **No** | **No** |

**401 vs 403 (te lo pueden preguntar)**

**401** = «no sé quién eres»: no enviaste el token o ya no sirve. **403** = «sé quién eres, pero no tienes permiso»: por ejemplo, un OPERADOR pidiendo el dashboard.

PARTE 3

# Lo que debes saber

## La estructura de clase: cada petición pasa por 4 capas

Igual que en clase: un archivo por entidad en cada capa. Ejemplo con una devolución:

| | |
|---|---|
| **routes** | `return_routes.py` · Dice la URL ( `POST /api/returns` ), exige el token ( `@jwt_required()` ) y el rol. No tiene lógica. |
| **controllers** | `return_controller.py` · Lee lo que llegó, valida que no falten datos (400) y llama al servicio. Responde el JSON. |
| **services** | `return_service.py` · Las reglas de negocio: ¿el objeto está DISPONIBLE?, ¿hay reclamación aceptada? Guarda todo en una transacción. |
| **models** | `return_record.py` · La tabla en Python (columnas, llaves, restricciones). |

Todas las respuestas tienen la misma forma: `{ "ok": true/false, "message": "...", "data": ... }`.

## Archivos clave del backend

| Archivo / carpeta | Qué es |
|---|---|
| `run.py` | Arranca la aplicación ( `python run.py` ). |
| `config.py` y `.env` | Configuración: dirección de la base, clave del JWT, orígenes permitidos (CORS). |
| `app/__init__.py` | `create_app()` : junta todo y registra las rutas con `/api` . |
| `app/extensions.py` | `db` , `migrate` y `jwt` . |
| `app/common/` | Ayudantes compartidos: formato de respuesta, errores, validación de fotos, PDF, fechas. |
| `migrations/versions/` | 8 migraciones: cada una crea un grupo de tablas. La última agrega los triggers anti-borrado. |
| `seed.py` / `seed_demo.py` | Datos iniciales (roles, sedes, 12 categorías, 129 tipos, ADMIN) / 24 objetos de demostración. |

## Las 15 tablas, agrupadas

| Grupo | Tablas |
|---|---|
| Cuentas y seguridad | `roles` , `users` , `password_reset_tokens` , `token_revocations` |
| Catálogos (lugares y tipos) | `campuses` (sedes), `blocks` , `departments` , `object_categories` , `object_types` |
| Personas | `persons` (quien entrega o reclama; no tienen cuenta) |
| Objetos | `object_lost` , `object_photos` |
| Proceso de devolución | `claims` (reclamaciones), `return_records` (devoluciones) |
| Historial | `audit_logs` (quién cambió qué, cuándo y por qué) |

## Conceptos clave en palabras simples

| Concepto | Explicación corta |
|---|---|

| | |
|---|---|
| JWT | Un «pase» que recibes al iniciar sesión. Cada petición lo lleva. El rol se lee de la base en cada petición, así que si el ADMIN te cambia el rol, aplica de inmediato. |
| Contraseñas | Nunca se guardan tal cual: se guarda un *hash* (con `werkzeug.security`, como en clase). |
| Recuperar contraseña | Llega un enlace con un código que dura 15 minutos y sirve una sola vez. |
| Llave foránea (FK) | Une tablas: un objeto apunta a su bloque, su tipo, su persona. Todas con `ON DELETE RESTRICT` (no se borra en cascada). |
| La sede sale del bloque | El objeto guarda el *bloque*; la sede se obtiene del bloque. Si se guardaran las dos, podrían contradecirse (3FN). |
| Nombre normalizado | « POBLADO » y «Poblado» se guardan como `poblado` en `normalized_name`, que es `UNIQUE`: no hay duplicados disfrazados. |
| Bloqueo optimista ( `version` ) | Si dos personas editan el mismo objeto, la segunda recibe 409 porque su versión quedó vieja; no pisa el cambio del otro. |
| Auditoría | Cada cambio guarda un renglón con «antes → después», en la misma transacción del cambio. |
| Migraciones | Scripts que crean o cambian las tablas en orden ( `flask db upgrade` ). |
| Seed | Datos iniciales. Se puede correr varias veces sin duplicar nada. |
| Fechas | Se guardan en UTC y se muestran en hora de Bogotá. |

PARTE 4 · REGLA DE NEGOCIO 1

# «Nada se borra»

El PDF lo exige: *«El backend impide eliminar físicamente filas, fotos e historial»*. Si alguien se equivoca, el registro se **inactiva con un motivo**, y queda la huella de quién lo hizo.

> **La idea en una frase**
> «En un sistema de custodia la información es evidencia: si un objeto se pierde de nuevo, necesito saber todo lo que pasó. Por eso nada se borra; los errores se marcan como inactivos y todo queda en el historial.»

## Paso a paso para explicarla (unos 3 minutos)

**1** **Di la regla y por qué existe.**

*«La primera regla es que nada se borra. Es un sistema de custodia: si borro un registro, pierdo la evidencia de quién recibió el objeto o a quién se le entregó.»*

**2** **Muestra que la API no tiene cómo borrar.**

*«En todo el backend no hay ninguna ruta DELETE. Para corregir hay rutas `/inactivate` y `/reactivate`.»*

**Qué mostrar:** en VS Code, abre `app/routes/object_lost_routes.py` y señala `PATCH /object-lost/<id>/inactivate`.

**3** **Muestra cómo se inactiva en la aplicación.**

*«Si un operador registró un objeto por error, lo inactiva, pero el sistema le exige un motivo.»*

**Qué mostrar:** en el frontend, abre el detalle de un objeto DISPONIBLE → Inactivar → escribe el motivo. El objeto sigue existiendo, ahora como inactivo.

**4** **Muestra el historial.**

*«Cada cambio queda en `audit_logs`: quién lo hizo, cuándo, el motivo y qué valor tenía antes y después. Se guarda en la misma transacción: si el cambio falla, tampoco queda el registro de auditoría, y viceversa.»*

**Qué mostrar:** el historial del objeto en el frontend, o en DBeaver: `SELECT entity, action, reason, occurred_at FROM audit_logs ORDER BY id DESC LIMIT 5;`

**5** **La prueba fuerte: intentar borrar directo en la base.**

*«Aunque alguien se salte la aplicación y entre directo a PostgreSQL, tampoco puede borrar. Pusimos triggers.»*

**Qué mostrar:** en DBeaver ejecuta:

```
DELETE FROM object_lost WHERE id = 1;
```

Resultado: **ERROR: No se permite eliminar registros de la tabla object_lost**. El registro sigue ahí.

**6** **Explica de dónde sale ese error.**

> *«Es la migración `8d6746a80648`. Crea una función `prevent_delete()` y un trigger en cada una de las 15 tablas que se dispara antes de cualquier DELETE o TRUNCATE. Además, `audit_logs` y `return_records` tampoco aceptan UPDATE, porque son historia.»*

**Qué mostrar:** el archivo `migrations/versions/8d6746a80648_add_triggers_that_block_delete_and_.py`.

**7** **Cierra con las otras protecciones.**

> *«Además, todas las llaves foráneas son `ON DELETE RESTRICT`, no hay cascadas. Y un objeto DEVUELTO ni siquiera se puede inactivar: es historia real.»*

**Si te preguntan…**

**«¿Y si de verdad hay que corregir algo?»** → Se edita con motivo (queda el antes y después), o se inactiva con motivo. Para objetos devueltos, el ADMIN tiene una «corrección administrativa» que corrige datos sin deshacer la devolución.

**«¿Para qué el trigger si no hay ruta DELETE?»** → La aplicación no es la única forma de llegar a la base (DBeaver, un script, otro programa). El trigger protege los datos desde la base misma.

PARTE 5 · REGLA DE NEGOCIO 2

# «Recepción con fotos segura y completa»

El PDF exige: de 1 a 5 fotos JPEG o PNG, máximo 5 MB cada una, **validando el contenido real**, y que la recepción sea **atómica**: si falla una foto, no puede quedar un registro a medias.

> **La idea en una frase**
> «Cuando se recibe un objeto, o se guarda todo (objeto + fotos + auditoría) o no se guarda nada; y solo se aceptan imágenes de verdad, no archivos disfrazados.»

## Paso a paso para explicarla (unos 3 minutos)

**1** **Di la regla.**

*«Cada objeto debe tener entre 1 y 5 fotos, JPEG o PNG, de máximo 5 MB. Y la recepción es todo o nada.»*

**2** **Muestra una recepción correcta.**

*«El operador llena el formulario, escoge sede y bloque, busca a la persona que entrega y adjunta las fotos.»*

**Qué mostrar:** en el frontend como OPERADOR, crea un objeto con 2 fotos. Señala que el código `OBJ-…` lo genera el servidor y que «Recibe» es el usuario que inició sesión.

**3** **El servidor no confía en el cliente.**

*«Quien recibe se toma del token, no del formulario, y el estado siempre arranca en DISPONIBLE. Postman lo prueba enviando `status = DEVUELTO` a propósito, y el servidor lo ignora.»*

**Qué mostrar:** en Postman, `11 ObjectLost → Exitosos → recepción con 2 fotos (PA08)`: en el Body está `status = DEVUELTO` y en las pruebas, «Estado DISPONIBLE (no el enviado)».

**4** **Prueba negativa: el archivo disfrazado.**

*«Validamos por contenido, no por extensión. Un ejecutable renombrado a `.jpg` se rechaza porque sus primeros bytes no son los de una imagen.»*

**Qué mostrar:** en Postman, la solicitud que sube `fotos/falso_ejecutable.jpg` → responde **400**: «no es una imagen JPEG o PNG válida». Si quieres, el de 6 MB → 400 por tamaño.

**5** **Explica cómo se reconoce una imagen real.**

*«Todo JPEG empieza con los bytes `FF D8 FF` y todo PNG con `89 50 4E 47`. Leemos esos primeros bytes; la extensión del nombre no importa.»*

**Qué mostrar:** `app/common/files.py`, función `detect_image_type`.

**6** **Explica el «todo o nada».**

> *«Primero se validan todas las fotos, antes de escribir nada. Luego se guardan el objeto, las fotos y la auditoría en una sola transacción. Si algo falla a mitad, se hace rollback y se borran los archivos temporales que alcanzaron a escribirse. A eso le llamamos compensación.»*

**Qué mostrar:** `app/services/object_lost_service.py`, función `create_object` (el `try` con el rollback) y `discard_uncommitted_files` en `files.py`.

**7** **Demuéstralo (opcional, prueba PA10).**

> *«Para demostrarlo simulamos que el disco falla al guardar la última foto.»*

**Qué mostrar:** con `ALLOW_FAULT_INJECTION=true` en el `.env` (y el servidor reiniciado), en Postman: `11 ObjectLost → Validaciones → fallo de archivo simulado` → responde **500**, y el objeto no aparece en el inventario. Después vuelve a dejarlo en `false`.

**8** **Cierra con la privacidad.**

> *«Las fotos se guardan con un nombre interno aleatorio, nunca con el nombre que mandó el usuario; en la base solo queda la referencia. Y se sirven por una ruta protegida: el rol CONSULTA no las puede ver.»*

**Si te preguntan…**

**«¿Se puede quedar un objeto sin fotos?»** → No. Si intentas inactivar la última foto activa, responde 409.
**«¿Por qué no confiar en la extensión?»** → Porque cualquiera puede renombrar un archivo. El contenido no miente.

PARTE 6

# Preguntas que te pueden hacer

**¿Qué relación de datos me puedes explicar?**
**Block 1:N ObjectLost, dos veces.** Un bloque tiene muchos objetos encontrados allí (`found_block_id`) y muchos objetos guardados allí (`custody_block_id`). Pueden ser bloques distintos: se encontró en el bloque A y se guarda en la oficina del bloque C.

---

**¿Por qué User y Person son tablas distintas?**
User es quien usa el sistema (tiene contraseña y rol). Person es quien entrega o reclama un objeto: un estudiante o visitante que no tiene cuenta.

---

**¿Qué diferencia hay entre ObjectType y ObjectLost?**
ObjectType es el catálogo («Celular»). ObjectLost es un objeto físico concreto: dos celulares perdidos son dos filas de `object_lost` con el mismo tipo.

---

**¿En qué forma normal está el modelo?**
Tercera forma normal: cada dato está una sola vez y en su tabla. Por ejemplo, la sede no se repite en el objeto porque ya se sabe por el bloque.

---

**¿Por qué el registro público crea siempre CONSULTA?**
Para que nadie se dé permisos a sí mismo. Solo un ADMIN puede subir a alguien a OPERADOR.

---

**¿Qué pasa si intento inactivar una sede que todavía tiene bloques activos?**
Responde 409 y explica el conflicto. Primero hay que inactivar sus bloques.

---

**¿Qué es una prueba negativa? Dame un ejemplo.**
Una prueba que verifica que el sistema **rechaza** algo incorrecto. Ejemplos: el ejecutable renombrado (400), CONSULTA pidiendo usuarios (403), una petición sin token (401), una sede duplicada (409), una fecha de hallazgo en el futuro (400).

---

**¿Cómo están organizadas las pruebas de Postman?**
18 carpetas, una por módulo. Cada una tiene subcarpetas **Exitosos**, **Validaciones** y **Permisos**. Las solicitudes se encadenan: el login guarda el token y los IDs creados se usan en las siguientes. Resultado: 781/781.

---

**¿Qué es una migración y por qué no crear las tablas a mano?**
Es un script versionado que crea o cambia tablas. Así cualquiera reconstruye la base igual con `flask db upgrade`, en orden, y queda historia de los cambios.

---

**¿Qué es propio de ustedes y no del enunciado?**
Las 4 rutas del dashboard y los supuestos S1–S10 del documento de análisis. No implementamos lo opcional de la sección 11 (control de ingreso de personas).

---

PARTE 7

# Chuleta de último minuto

## Credenciales (ficticias)

| Usuario | Clave | Rol |
|---|---|---|
| `admin@ces.test` | `Admin2026!` | ADMIN |
| `operador.demo@ces.test` | `Demo2026!` | OPERADOR |
| `consulta.demo@ces.test` | `Demo2026!` | CONSULTA |

## Para arrancar todo

```
cd backend
.venv\Scripts\activate
python run.py            # API en :5000

cd frontend
npm start                #
http://localhost:4200
```

## Códigos HTTP

| Código | Significa |
|---|---|
| 200 / 201 | Bien / creado |
| 400 | Datos inválidos o faltantes |
| 401 | Sin token o token inválido |
| 403 | Sin permiso para ese rol |
| 404 | No existe |
| 409 | Conflicto con una regla (duplicado, ya devuelto, versión vieja) |
| 429 | Demasiados intentos |

## Las 3 cosas que cada integrante debe poder explicar (lo pide el PDF)

| Tipo | Ejemplo listo |
|---|---|
| Regla de negocio | «Nada se borra» (parte 4) o «Recepción con fotos segura y completa» (parte 5) |
| Relación de datos | Block 1:N ObjectLost dos veces (hallazgo y custodia); la sede se obtiene del bloque |
| Prueba negativa | Ejecutable renombrado a `.jpg` → 400; CONSULTA pidiendo `/users` → 403 |

**Si algo falla en vivo**

Mantén la calma y muestra la evidencia guardada: la captura de Postman (781/781), la salida de pytest (55 passed) y la de concurrencia. El PDF valora que no se afirme nada sin evidencia, y tú la tienes.

**Antes de entrar**

Backend y frontend corriendo · DBeaver conectado · Postman abierto en la colección · VS Code con el proyecto · `ALLOW_FAULT_INJECTION=false` salvo que vayas a mostrar PA10.